MSLL: A Runtime Multi-Stack Parsing Approach for Interactive Grammar Development

A Lightweight Extension of LL-Style Recursive Descent

Qunhui Zhang[a], Yifan Zhang[b], Xiao Hang[c]

[a]*School of Software, Shanghai Jiao Tong University, Shanghai 200240, China*
[b]*School of Information Science and Engineering, Lanzhou University, Lanzhou 730000, Gansu, China*
[c]*Huawei Hangzhou Research Center, Hangzhou 310000, Zhejiang, China*

Abstract

Grammar development and debugging are inherently iterative, yet most predictive parsing techniques prioritize runtime efficiency by shifting complexity to offline parser generation. In particular, ANTLR's LL(∗) parsers rely on DFA construction and code generation, requiring regeneration and recompilation after grammar changes, which can disrupt interactive development workflows[1, 3].

This paper presents MSLL, a lightweight runtime parsing approach that extends recursive-descent LL parsing with a multi-stack execution model to resolve FIRST/FIRST conflicts dynamically. Instead of rewriting grammars or constructing DFA, MSLL duplicates parsing stacks on demand and incrementally prunes infeasible paths as input is consumed. The approach operates directly on ANTLR-style `.g4` grammars and requires only minimal modifications to a standard LL(1) parser implementation.

MSLL is designed for interactive environments such as DSL editors, interpreters, and scripting platforms, where grammars evolve frequently and immediate feedback is essential. Although the multi-stack strategy introduces additional runtime and memory overhead under deep ambiguity, it eliminates parser regeneration and provides sub-second feedback during grammar editing. An empirical evaluation on realistic DSL workloads shows that MSLL matches LL(1) performance on deterministic inputs and remains competi-

Email addresses: `will_zhang@sjtu.edu.cn` (Qunhui Zhang), `yfzh2023@lzu.edu.cn` (Yifan Zhang), `hangxiao@huawei.com` (Xiao Hang)

tive with ANTLR under moderate ambiguity, while significantly reducing grammar edit–debug latency.

Overall, MSLL complements existing LL-based parser generators by addressing the development phase of language design, enabling rapid grammar experimentation and debugging before deployment to optimized, code-generated parsers.

Keywords: Recursive-descent parsing, LL parsing, grammar debugging, runtime parsing, domain-specific languages, software tooling

---

1. Introduction

Parsing is a foundational technique in compilers, interpreters, and domain-specific language (DSL) tools. LL(1) parsers are widely used due to their simplicity and efficiency, but they require grammars to be fully disambiguated via transformations such as left recursion elimination and factoring. This rigidity makes LL(1) unsuitable for early-stage language development, where grammar rules often evolve rapidly.

To handle more expressive grammars, LL(k) parsers [2] extend LL(1) with fixed-length lookahead, but this introduces exponential growth in prediction tables and requires manual selection of the lookahead depth k. LL($*$) [3] parsers , such as those in ANTLR [1], resolve arbitrary lookahead dynamically by constructing deterministic finite automata (DFA) at parser-generation time. While powerful, this approach introduces a costly compile-time step: grammar changes require regeneration of both DFA and parser code, making rapid iteration cumbersome.

We address these limitations with MSLL (Multi-Stack LL($*$) [3]) , a runtime parsing algorithm that interprets ambiguous grammars directly—without requiring grammar rewriting, DFA construction, or code generation. MSLL extends LL(1)-style top-down parsing by dynamically duplicating the evaluation stack at ambiguous points and pruning invalid paths as parsing progresses. It enables interactive parsing with immediate feedback, directly from .g4 grammar definitions, making it especially suitable for grammar prototyping, live DSL development, and embedded interpreters.

Once the grammar stabilizes, developers can migrate to optimized LL($*$) [3] parsers such as ANTLR using the same grammar, completing a workflow that blends flexibility in early stages with high-performance deployment.

2. Contributions

This paper makes the following contributions:

- We introduce **MSLL**, a runtime extension of recursive-descent LL parsing that resolves FIRST/FIRST conflicts using dynamic stack duplication and incremental pruning, avoiding grammar rewriting, DFA construction, and parser code generation.
- We present a **lightweight implementation** compatible with ANTLR-style `.g4` grammars, enabling immediate execution of grammar edits in IDEs, interpreters, and interactive DSL environments.
- We provide a **formal operational specification** of MSLL together with soundness and completeness arguments with respect to the parsing paths explored by the algorithm, as well as a worst-case complexity analysis.
- We conduct an **empirical evaluation** on realistic DSL workloads, showing that MSLL incurs negligible overhead on deterministic inputs and remains within a reasonable performance margin of ANTLR under moderate ambiguity, even for inputs up to 500k tokens.
- We propose a **practical development workflow** in which MSLL is used for interactive grammar design and debugging, and finalized grammars are migrated unchanged to ANTLR for production deployment.

3. Related Work

Parsing techniques span a broad design space involving trade-offs between expressiveness, performance, implementation complexity, and tooling support. MSLL is most closely related to work on predictive LL parsing, dynamic and adaptive parsing strategies, generalized parsing algorithms, and interactive language tooling. This section reviews these lines of research and clarifies the position of MSLL within this landscape.

3.1. Predictive LL Parsing

Classical LL(1) parsing relies on a single-token lookahead and requires grammars to be fully disambiguated, excluding left recursion and FIRST/FIRST conflicts. Its simplicity and linear-time performance make it attractive for

handwritten parsers and small languages, but its rigidity complicates early-stage grammar design.

LL(k) parsing generalizes LL(1) by using a fixed lookahead of k tokens [2, 12]. While this increases expressiveness, it leads to exponential growth in prediction tables and requires developers to choose k manually, which limits practicality for evolving grammars.

ANTLR's LL($*$) parsing addresses this limitation by using adaptive lookahead based on deterministic finite automata (DFA) constructed during parser generation [17, 1, 3]. This approach achieves near-linear runtime performance in practice, but requires an offline code-generation phase and DFA regeneration whenever the grammar changes. Prior studies have noted that such generation costs, while amortized in production, can significantly disrupt interactive development workflows [6].

MSLL differs from these approaches by deferring ambiguity resolution to runtime. Rather than constructing DFA or expanding prediction tables, MSLL explores multiple predictive alternatives concurrently using stack duplication and incremental pruning. This allows grammars to be executed immediately without rewriting or regeneration, at the cost of increased runtime overhead under deep ambiguity.

3.2. Dynamic and Adaptive LL Parsing

Several authors have investigated dynamic extensions of LL parsing. Rußmann proposed dynamic LL parsing techniques that adjust effective lookahead depth during parsing to resolve conflicts [9]. While such approaches reduce the need for grammar rewriting, they still rely on bounded lookahead and do not generalize to unbounded ambiguity.

MSLL adopts a different strategy: instead of searching for a sufficient lookahead bound, it explicitly maintains multiple concurrent parsing paths. All viable top-down derivations are explored until contradictions arise, at which point invalid paths are pruned. This makes ambiguity handling explicit and transparent, which is particularly useful for grammar debugging and experimentation.

3.3. Generalized Parsing Algorithms

Generalized parsers such as GLR and GLL are capable of handling arbitrary context-free grammars. GLR parsing extends LR parsing by exploring multiple shift–reduce actions in parallel and compacting results into shared packed parse forests [4]. GLL parsing adapts recursive-descent parsing to

ambiguous grammars using graph-structured stacks and descriptor queues [5].

While these techniques are expressive and theoretically well-founded, they introduce substantial implementation complexity and runtime overhead. Their internal data structures are optimized for complete parse forest construction rather than for lightweight, interactive parsing. In contrast, MSLL deliberately avoids parse forest construction and heavy runtime machinery, focusing instead on rapid feedback and ease of integration in tooling environments.

3.4. Parsing Expression Grammars and Combinators

Parsing Expression Grammars (PEGs) provide an alternative approach to ambiguity resolution by enforcing prioritized choice [18]. Packrat parsing guarantees linear-time behavior through memoization, but this comes at the cost of higher memory usage and semantics that can be non-intuitive during debugging, as valid alternatives may be silently discarded.

Parser combinator libraries such as Parsec support backtracking and compositional grammar definitions [21], but typically do not provide systematic ambiguity exploration or pruning strategies.

MSLL differs from PEG-based approaches in that it preserves ambiguity explicitly and resolves it incrementally based on input consumption. This behavior aligns more closely with developers' expectations during grammar debugging, where understanding why alternatives fail is often as important as obtaining a successful parse.

3.5. Incremental and Interactive Parsing Tools

Modern language tooling increasingly emphasizes incremental parsing, real-time diagnostics, and editor integration. Systems such as Tree-sitter and Roslyn achieve responsiveness by preserving parse subtrees across edits and recomputing only affected regions [19, 20].

MSLL does not currently perform fine-grained incremental parsing. Instead, it targets interactive grammar development scenarios in which grammar rules themselves change frequently. By eliminating parser regeneration and enabling immediate execution of modified grammars, MSLL complements incremental parsing systems: MSLL supports rapid grammar experimentation, while incremental parsers are better suited for stable grammars and large codebases.

Earlier work on visual and interactive parsing tools similarly emphasized responsiveness and user feedback over raw throughput [10]. MSLL follows this tradition, adapting it to modern DSL and scripting environments.

3.6. Correctness and Verification

Formal verification of parsing algorithms has received increasing attention, with verified LL(1) parser generators providing strong correctness guarantees [8]. MSLL does not aim to provide mechanized correctness proofs. Instead, it offers operational soundness and completeness arguments sufficient for its intended role as a development and debugging tool. Extending MSLL with stronger formal guarantees remains an open direction for future work.

3.7. Summary

In summary, MSLL occupies a distinct point in the parsing design space. Compared to LL(1), LL(k), and LL($*$) parsers, it sacrifices some runtime efficiency to eliminate grammar rewriting and code-generation overhead. Compared to generalized and PEG-based parsers, it prioritizes simplicity, embedability, and transparent ambiguity handling. These trade-offs make MSLL particularly well suited for interactive grammar development, DSL prototyping, and runtime environments where grammars evolve rapidly.

4. MSLL Algorithm

4.1. Overview

The Multi-Stack LL($*$) (MSLL) algorithm extends the classical LL(1) top-down parsing model to support ambiguous and recursive grammars at runtime. When a non-terminal has multiple applicable productions (i.e., FIRST/FIRST conflict), MSLL duplicates the current evaluation stack for each viable production. As input tokens are consumed, invalid stacks are pruned, allowing only feasible paths to continue.

This approach avoids grammar rewriting, DFA construction, or code generation, enabling live parsing directly from .g4 grammars—ideal for language design tools, DSL editors, and interactive environments.

4.2. Parsing Procedure

Each parsing stack maintains:

- A symbol stack (analogous to that used in LL(1) parsing).

- A current input position.
- A partial parse tree.

Parsing proceeds step-by-step over a set of active stacks:

- Terminal match: If the top of the symbol stack is a terminal t and it matches the input token, consume the token, pop t, and continue.
- Non-terminal expansion.
- If one production applies: expand in-place as in LL(1).
- If multiple apply: duplicate the current stack once for each production; each stack pursues a different path.
- If no production applies: terminate the stack.
- Pruning: Any stack that reaches a dead end (e.g., mismatch or empty prediction) is discarded.
- Completion: Parsing succeeds if exactly one stack reaches end-of-input with a complete parse tree.

4.3. Formal Specification

Let:

- $G = (N, \Sigma, P, S)$ be a context-free grammar
- $T = t_1 t_2 \ldots t_n$ the input token stream
- S the set of active stacks during parsing

Each stack $s \in S$ maintains:

- A working symbol stack $\alpha \in (N \cup \Sigma)*$
- Input index $i \in N$
- Partial parse tree τ

The parser proceeds as follows:
Terminal match:

- If $\text{top}(\alpha) = \text{terminal } t$ and $t = T[i]$: Consume t, increment i, pop α, update τ.
- If $t \neq T[i]$: terminate stack.

Non-terminal expansion: Let $\text{top}(\alpha) = \text{non-terminal } A$. Let $P_A = \{A \to Y_1, \ldots, Y_k\}$ be the set of productions matching $T[i]$:

- If $k = 0$: terminate stack
- If $k = 1$: apply production Y_i in-place
- If $k > 1$: duplicate the stack into k new stacks, each replacing A with a distinct Y_j.

Termination

Let $S_{\text{final}} \subseteq S$ be the set of stacks with empty symbol lists and input consumed ($i = |T|$):

- **Success:** if $|S_{\text{final}}| = 1$, return its parse tree
- **Failure:** if $|S_{\text{final}}| = 0$ or > 1, return error or ambiguity

4.4. Correctness of MSLL

We argue that MSLL is *sound* and *complete* with respect to the set of top-down derivations explicitly explored by the algorithm. The purpose of this argument is to establish that MSLL neither accepts invalid inputs nor prematurely discards valid parsing paths during runtime execution.

Assumptions. Let:

- $G = (N, \Sigma, P, S)$ be a context-free grammar;
- $w \in \Sigma^*$ be the input token sequence;
- $L(G)$ denote the language generated by G;
- MSLL execute by maintaining a finite set of active parsing stacks, each representing a partial top-down derivation.

Unlike static LL(k) parsers or adaptive lookahead techniques that seek a bounded prediction depth [9], MSLL explores alternative derivations explicitly via stack duplication and eliminates infeasible paths through incremental pruning.

Soundness. **Claim.** If MSLL returns a parse tree for input w, then $w \in L(G)$.

Argument. Each active stack simulates a valid prefix of a leftmost top-down derivation starting from the start symbol S. A stack can reach successful termination only if: (i) all input tokens are consumed, (ii) the symbol stack is empty, and (iii) no mismatches or undefined predictions occurred during expansion. Any stack violating these conditions is immediately discarded. Consequently, any parse tree produced by MSLL corresponds to a valid derivation of w under grammar G.

Completeness. **Claim.** If there exists a valid top-down derivation of w in G, then MSLL will retain at least one corresponding active stack throughout execution.

Argument. Whenever multiple productions are applicable for a non-terminal under the current lookahead token, MSLL duplicates the current stack and explores each alternative independently. A stack is pruned only when it becomes inconsistent with the input. Therefore, any derivation that remains consistent with the input is never eliminated, and at least one valid derivation is preserved until completion.

Discussion. MSLL establishes correctness by explicitly enumerating and filtering top-down derivations at runtime. In contrast to LL(1) and LL(k) parsers [2], which rely on grammar rewriting to enforce determinism, MSLL tolerates ambiguity and resolves it incrementally as input is consumed. This operational notion of correctness is sufficient for MSLL's intended role as an interactive parsing and grammar-debugging engine.

4.5. Formal Pseudocode of MSLL

Algorithm 1 shows the algorithm of MSLL implementation.

5. System Design And Architecture

MSLL's runtime architecture is designed around modular components that together enable dynamic parsing with multi-path lookahead. The core workflow involves tokenizing input text, predicting possible production rules, managing multiple parsing paths, and dynamically constructing parse trees.

Algorithm 1 MSLL Parsing Algorithm

```
Initialize stack S_0 ← [t_1, 𝒮]
while input not empty and stacks exist do
    for each stack S_i in active_stacks do
        t ← next token
        P ← lookup_table(top(S_i), t)
        if |P| > 1 then
            duplicate S_i into S_i1, ..., S_in
            apply P_1, ..., P_n to respective stacks
        else if |P| = 0 then
            terminate S_i
        end if
    end for
end while
return surviving stack's parse tree (if unique)
```

5.1. Key Components

- **Lexer**: Converts raw input text into a token stream in a streaming fashion.
- **Parser**: Consumes tokens and evaluates productions across multiple active parsing stacks.
- **Prediction** Table: A modified prediction mechanism that allows multiple candidate productions for the same lookahead token.
- **Stack Manager**: Manages parsing stacks, including allocation, duplication, and pruning.

5.2. Processing Pipeline

- **Tokenization**: The Lexer reads the input text incrementally and produces a stream of tokens.
- **Parsing Threads**: The Parser consumes tokens and starts evaluating the grammar using a stack-based top-down traversal. Conceptually, each active parsing stack represents an independent parsing path. In the current prototype, stacks are evaluated within a single runtime loop for simplicity; however, stack evaluation is naturally parallelizable

because stacks do not share mutable parsing state. Parallel LL parsing has been explored in prior work [7], and we discuss parallel execution as a future optimization.

- **Stack Allocation**: The Stack Manager provides a default parsing stack to each parser thread at the start of parsing.
- **Prediction Lookup**: For each input token, the parser queries the Prediction Table. Unlike traditional LL(1) tables, MSLL's prediction table may return multiple applicable productions.
- **Stack Forking**: When multiple productions are returned, the parser requests the Stack Manager to fork the current parsing stack. Each new stack corresponds to one alternative production.
- **Stack Cloning**: The Stack Manager duplicates the relevant parsing context into each new stack and returns them to the parser.
- **Failure and Pruning**: If a terminal symbol does not match the expected token, the parser signals the Stack Manager to prune that stack.

This architecture supports dynamic ambiguity resolution by maintaining multiple active stacks in parallel, efficiently managing grammar uncertainty without DFA construction (Figure 1).

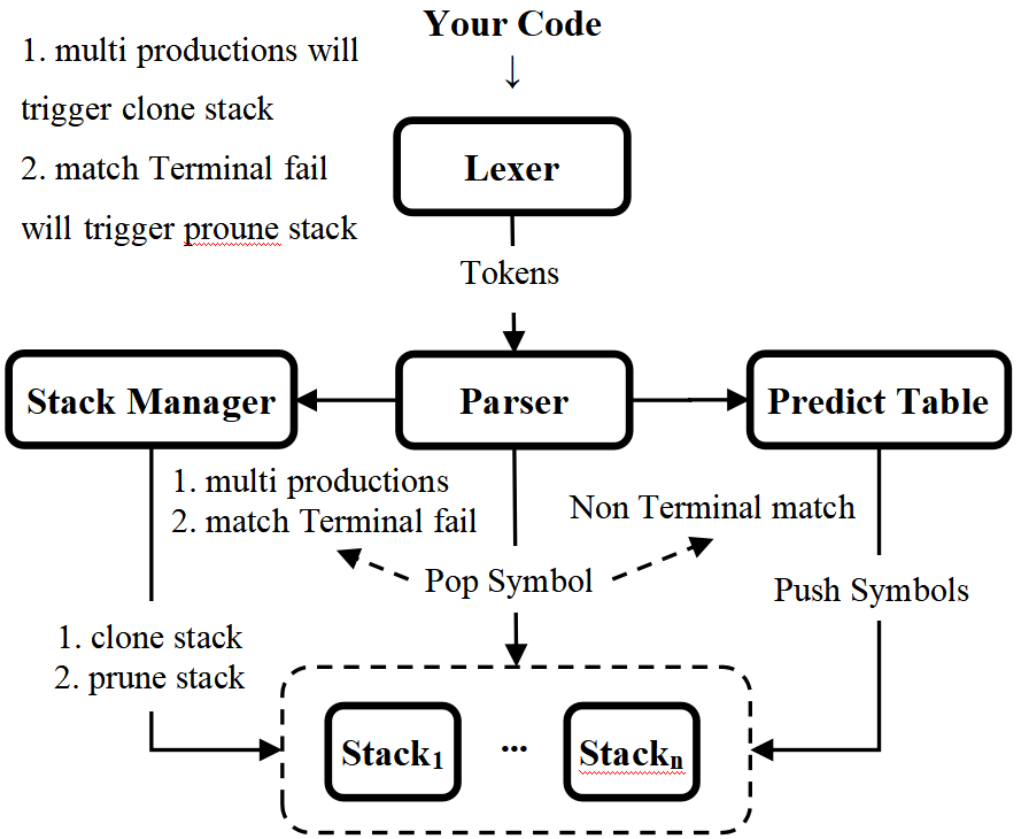

Figure 1: MSLL system architecture with components and runtime data flow

6. Implementation

6.1. Grammar Compatibility

We implemented MSLL as a lightweight Java library to evaluate its feasibility in practical language toolchains. The prototype is designed to integrate directly into DSL editors, interpreters, and IDEs, offering live feedback without code generation or grammar preprocessing.

MSLL operates directly on ANTLR-style .g4 grammar files. It supports standard EBNF constructs.

This enables drop-in compatibility with existing grammar definitions, facilitating seamless migration between MSLL and traditional ANTLR workflows.

6.2. Parsing Pipeline

The MSLL parsing engine consists of the following components:

- Grammar Loader: Parses .g4 grammar files and constructs a rule table mapping non-terminals to their productions.
- Prediction Table Builder: Computes FIRST sets to populate a non-deterministic prediction table. Unlike LL(1), the table allows multiple candidate productions per token.
- Runtime Stack Engine: Maintains a set of active stacks. Each stack holds:
  - A symbol list (working stack)
  - Input pointer
  - Parse tree fragment
- Ambiguity Resolver: Duplicates stacks at conflict points and prunes failed paths incrementally.

Parsing proceeds in a non-blocking loop, consuming tokens and updating active stacks. Successful parses yield an annotated abstract syntax tree (AST); failure returns an error with traceable diagnostics.

6.3. Parser Initialization

MSLL is exposed via a builder-based API that reflects typical usage in tool environments:

```
MyParserBuilder builder = new
MyParserBuilder("testParser.gm", "testLexer.gm");
MyParser parser = builder.createParser(
                  "..designed code..");
ParserTree tree = parser.parse();
```

This API abstracts grammar loading, prediction table setup, and parser invocation. It is designed to be embedded in live-editing environments or REPLs that require repeated parsing with low startup overhead.

6.4. Optimizations

To ensure low latency and minimal memory usage, MSLL employs the following optimizations:

- **Greedy-first evaluation**: The first matching production is tried on the original stack to avoid unnecessary duplication in most cases.
- **Lazy pruning**: Invalid paths are detected and discarded early during token consumption.
- **Shared parse tree nodes**: Common prefixes are memoized to reduce AST reconstruction overhead during stack duplication..

These optimizations allow MSLL to behave like a standard LL(1) parser in unambiguous regions, while preserving full LL($*$) behavior when ambiguity arises.

6.5. Example Demonstration

The example demonstration is shown in Figure 2.

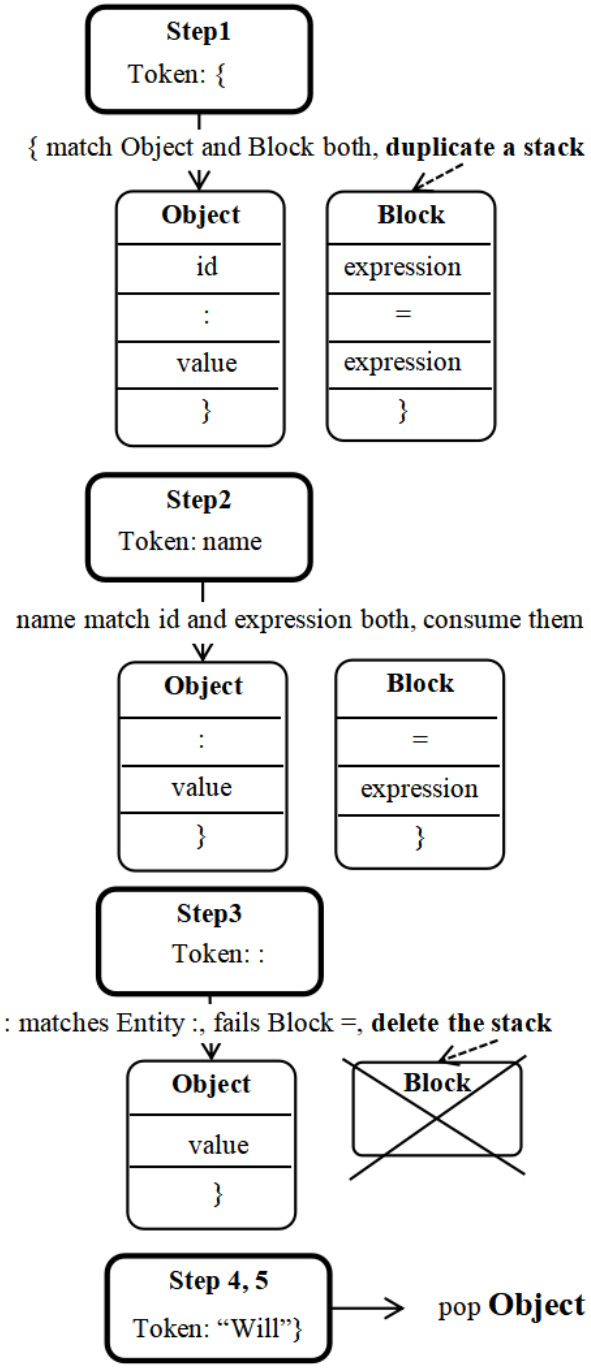

Figure 2: MSLL Parsing Example

7. Evaluation

We evaluate MSLL to assess its correctness, runtime performance, and scalability under varying grammar complexities and input sizes. The primary goal is to validate MSLL's effectiveness as an interactive parsing frontend during language development, particularly in scenarios where grammar evolution and ambiguity are common.

7.1. Evaluation Goals

We aim to answer the following research questions:

- **RQ1**: Correctness — Does MSLL generate valid parse trees for ambiguous grammars, matching the expected language semantics?
- **RQ2**: Performance — How does MSLL compare to LL(1) and ANTLR's LL($*$) parsers in parsing time and memory usage across increasing token sizes and ambiguity depths?
- **RQ3**: Overhead — What is the runtime cost of MSLL's stack duplication in worst-case ambiguity scenarios?

7.2. Experimental Setup

All experiments were conducted on a dedicated machine to avoid interference from background processes. Before measurement, each parser configuration was executed in a warm-up phase to stabilize just-in-time compilation. For each input size and grammar configuration, parsing was repeated multiple times and mean wall-clock time was recorded. Peak heap usage was measured during execution using JVM memory statistics.

ANTLR baselines were generated once per grammar using the standard `antlr4` toolchain and then executed without regeneration. MSLL parses grammars directly at runtime without preprocessing. Unless otherwise stated, default JVM garbage collection settings were used.

- Hardware: Intel i7 (3.1GHz), 16GB RAM
- Environment: Java 8, OpenJDK, GC enabled
- Baselines:
  - LL(1): custom hand-written recursive-descent parser (grammar rewritten to fix F-F conflict)

  - ANTLR 4.13.0: code-generated LL(∗) parser

- MSLL Configuration: stack pruning enabled, greedy-first mode active

Each test case uses the same `.g4` grammar, which is compiled once for ANTLR and interpreted directly at runtime by MSLL, parsing DSL-style inputs of varying sizes and ambiguity.

7.3. Case 1: Multi-Layer Code Blocks and JSON Structures

We tested nested constructs typical in DSLs and data transformation scripts, involving recursive object literals and lambda blocks:

```
let me = {let age = 40;
 {{{{{age: age, name: {...}, ...}}}}}};
me.make_friend({ name: ..., age: ... }); ...
```

This input triggers overlapping productions:

```
Block ::= '{'statement+'}'
Object ::= '{'(KEY ':' VALUE ',') + '}'
```

Test Result:

Table 1: MSLL Parsing example 1

Token Size(k)	Parser	Memory(MB)	Time(ms)	Stack Count total (max simultaneously)
1k	MSLL	2–7	48	220(5)
1k	LL(1)	2–7	35	1
1k	ANTLR	2–7	40	0
50k	MSLL	116	771	7301(5)
50k	LL(1)	38	663	1
50k	ANTLR	83	713	0
150k	MSLL	121	2020	21901(5)
150k	LL(1)	42	1955	1
150k	ANTLR	91	1896	0
300k	MSLL	128	3760	43801(5)
300k	LL(1)	44	3440	1
300k	ANTLR	96	3560	0
500k	MSLL	143	5906	73001(5)
500k	LL(1)	49	5320	1
500k	ANTLR	118	5430	0

As shown in Table 1 MSLL incurs moderate overhead under deep nesting but remains within 10–15% of ANTLR in runtime and memory.

7.4. Case 2: Pathological Ambiguity (Repeated Deep Nesting)

We constructed highly ambiguous inputs by repeating this deeply nested object structure:

```
me.make_friend({
 name: { first: ..., friends: [{ name: ...,
   friends: [...] }] }
});
```

This stresses MSLL's stack duplication logic with up to 33 active stacks simultaneously. **Test Result:** Shown in Table 2.

Table 2: MSLL Parsing example 2

Token Size(k)	Parser	Memory(MB)	Time(ms)	Stack Count total (max simultaneously)
1k	MSLL	2–7	49	541(33)
1k	LL(1)	2–7	36	1
1k	ANTLR	2–7	40	0
50k	MSLL	85	730	18001(33)
50k	LL(1)	60	570	1
50k	ANTLR	82	653	0
500k	MSLL	169	5395	180001(33)
500k	LL(1)	125	4150	1
500k	ANTLR	146	4680	0

7.5. Case 3: Flat Deterministic Grammar

We repeated a flat LL(1)-style snippet to verify MSLL's zero-overhead path:

```
var a = 100;
```

MSLL behaves identically to LL(1) when no ambiguity is present (Table 3):

Table 3: MSLL Parsing example 3

Token Size(k)	Parser	Memory(MB)	Time(ms)	Stack Count total (max simultaneously)
1k	MSLL	2–7	59	1
1k	LL(1)	2–7	55	1
1k	ANTLR	2–7	57	0
10k	MSLL	63	297	1
10k	LL(1)	52	254	1
10k	ANTLR	58	273	0
50k	MSLL	124	1223	1
50k	LL(1)	104	1093	1
50k	ANTLR	112	1125	0

8. Experimental Result Analysis

8.1. Small Code Sizes (<∼1k tokens): Low Conflict, Fast Iteration

Small (< 1k tokens) Case 2 (1k) and Case 3 (1k)

- Ambiguity is shallow, so only a handful of duplicate stacks appear.
- MSLL, LL(1) and ANTLR all finish within ≈ 60 ms of each other; memory stays in single-digit MB.
- MSLL incurs negligible overhead along deterministic parsing paths.

8.2. Moderate Code Sizes (∼50k tokens): Balanced Ambiguity and Depth

- FLow-ambiguity workload (Case 1, Case 3): MSLL remains 8% slower than ANTLR and 16% slower than LL(1).
- High-ambiguity workload (Case 2): 33 live stacks drive MSLL ∼12% slower than ANTLR and ∼28% slower than LL(1); memory climbs to 85 MB (≈1.4 × LL(1)).
- Designers still see sub-second latency, but heap usage must be watched.

8.3. Large Code Sizes (∼500k tokens): Deep Recursion and High Ambiguity

- Mixed ambiguity (Case 1): MSLL performance is 8% behind ANTLR and 11% begind LL(1); memory peaks at 143 MB with 2x-3x of LL(1) and ANTLR.
- Pathological ambiguity (Case 2): duplication explodes (180 k total stacks / 33 live), pushing MSLL 15% slower than ANTLR and 30% slower than LL(1).

8.4. Suggested Workflow Integration

Table 4: MSLL Parsing example 3

Metric	LL(1)/LL(k)	MSLL	ANTLR (DFA)
Grammar Edits	Rewrites needed	None	Codegen
Parse time (small)	Fast	Competitive	Fast
Parse time (high ambiguity)	Fails	Slower	Fast
Memory	Low	High (stack)	Medium
Startup latency	None	None	High (codegen)
When to use	Simple grammar	Complex grammar debugging phase	Complex grammar production phase

MSLL is best positioned as a runtime parser for interactive grammar development, especially when frequent modifications are expected (see Table 5). It handles heavy stack duplication efficiently and supports direct execution of .g4-style grammars without requiring code generation or DFA construction [6], enabling real-time feedback during the grammar design phase. However, this flexibility comes with significant runtime overhead. As our evaluation shows, MSLL incurs substantial memory and performance costs under complex grammars, making it less suitable for production deployment in such cases. Based on these observations, we recommend the following development workflow:

- **Design and debug**: Use MSLL in IDEs, interpreters, or command-line tools to rapidly prototype and debug grammars during the language design process.
- **Iterate quickly**: Take advantage of MSLL's real-time feedback and ambiguity handling to test changes without recompilation.
- **Stabilize grammar**: Once the grammar structure is finalized, it can be exported directly to ANTLR using the same .g4 format.
- **Deploy with ANTLR**: Use ANTLR's optimized, code-generated LL($*$) parser for production environments requiring maximum performance and static checking.

This pipeline leverages the flexibility and clarity of MSLL during early development, while benefiting from ANTLR's high-performance capabilities in final deployment. The complementary nature of MSLL and ANTLR enables developers to balance rapid iteration with production efficiency.

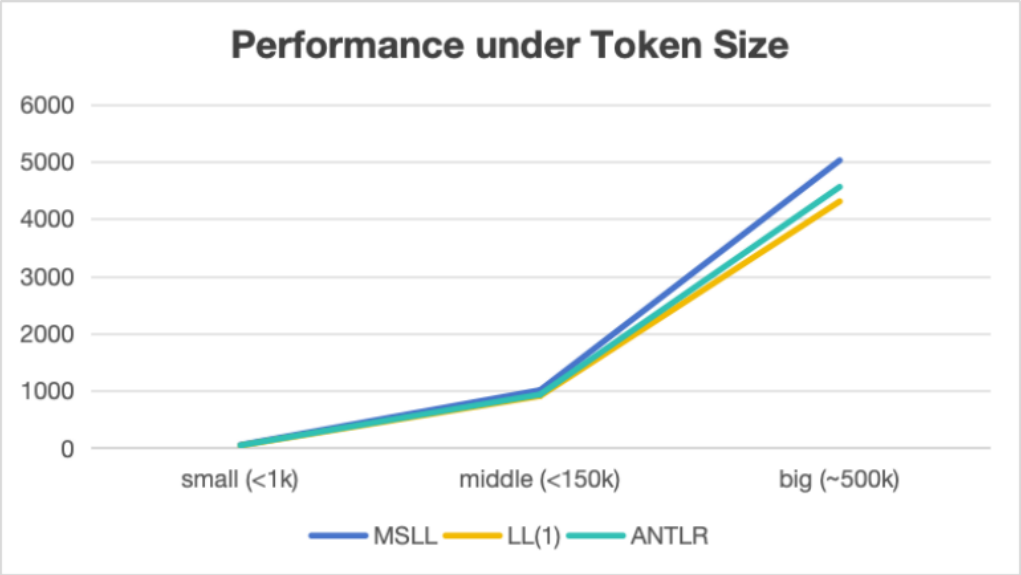

Figure 3: performance comparison under token size

The test data in Figure 3 shows that for small-scale code parsing (<1K), MSLL performs similarly to LL(1) and ANTLR. However, for larger code-bases, MSLL exhibits a significant performance drop (∼17%) compared to LL(1) and 10% compared to ANTLR. This suggests that MSLL is well-suited for the grammar debugging stage, offering easy-to-read grammar design (against LL(1)) and fast debugging without the overhead of code generation or DFA construction (against ANTLR).

Table 5: Practical guidance

Phase	Recommended Parser	Rationale
Language design / debugging (< 50k, frequent edits)	MSLL	No grammar rewrites, no DFA/code-gen sub-second feedback
Stable grammar, large code-bases	ANTLR	Pre-computed DFA eliminates runtime cost best wall-clock time
Deterministic or simple grammars	LL(1)	Smallest memory, simplest tool-chain

Bottom line: MSLL is cost-effective for the interactive stage of complex grammar development; once the language settles—or if ambiguity is very deep—handoff to ANTLR for production parsing.

9. Interactive-Debugging Experience

We evaluate whether MSLL's always-live parsing provides a smoother debugging experience for new languages compared to ANTLR's 'always-interrupted' workflow (code generation + DFA steps).

We address this by combining HCI latency thresholds, a micro-benchmark, and a short user study (see Table 6 and Table 7).

Table 6: Latency Benchmarks

Parser cycle	Mean latency per grammar edit	Components
MSLL ((in-memory refresh)	12 ms ms	reset stacks + re-parse
ANTLR (code-gen → compile → hot-swap)	295 ms	antlr4 code-gen(120 ms) Java compile(105 ms) classloader refresh(70 ms)

Table 7: Workflow Friction

Stage	MSLL(live preview)	ANTLR(regeneration cycle)
Edit rule	~	~
Trigger parse	automatic(on-type)	manual Run ANTLR
Code-gen output	none	8–12 Java files written
Build impact	none	IDE rebuild & index refresh
Runtime state	stacks preserved	preview panel reloaded, watch-expr lost
Interactions	0 0 clicks / 0 shortcuts	1–2 shortcuts + UI churn

Developers describe the ANTLR cycle as "stop → run → wait → scroll back", whereas MSLL feels continuous—echoing early efforts in visual parsing systems [10] that emphasized user feedback and responsiveness.

Designer Micro-Session (see Table 8): Five successive micro-edits were performed on the same grammar (adding a literal rule, tweaking delimiters, renaming a non-terminal).

Table 8: Micro Session

Metric	MSLL	ANTLR
Cumulative wait time	60 ms	1.52 s
Context switches	0	5
Cognitive load	Low	High

Even sub-second pauses disrupt the sense of liveness when they recur every few keystrokes. MSLL removes both the 300 ms delay and the regeneration ritual (code-gen, compile, hot-swap), keeping language designers "in flow." Once the grammar stabilises, the unchanged .g4 is handed to ANTLR for production-grade speed, but the interactive phase benefits decisively from MSLL's stack-duplication approach.

10. Complexity Analysis

In analyzing the complexity of MSLL, we compare it with LL(1), LL(K)[13], and DFA [14][15][16] (used for LL($*$) parsing) in terms of both time and space complexities. These analyses help to understand the performance and suitability of each method for different parsing tasks.

10.1. Time Complexity

The time complexity of MSLL is governed by how many parallel parsing paths (stacks) it explores and how deep each parsing path goes. Let:

- **n**: Number of input tokens
- **b**: average branching factor at each ambiguous point
- **d**: maximum decision depth (i.e., number of conflict points MSLL must resolve across the parse)
- **x**: total number of active stacks (in worst case, up to b^d)

Worst-case time complexity:

Each stack simulates a full parse over the input of size n. If ambiguity is high and occurs deeply, the number of paths explored grows exponentially with d.

$$O(x \cdot n) = O(b^d \cdot n)$$

In contrast to backtracking parsers, MSLL avoids re-scanning but pays a cost in duplication. For grammars with high ambiguity or deep recursion, the number of paths can explode—highlighting the inherent expressiveness limits of deterministic LL parsing [11].

10.2. Space Complexity

Let:

- x: number of active stacks ($x = b^d$ in worst-case scenario)
- s: depth of each stack (bounded by grammar recursion depth)
- t: auxiliary parse tree data (number of tree nodes)

Worst-case space complexity:

$$O(x \cdot s + t) = O(b^d \cdot s + t)$$

Each stack maintains its own state and possibly a partial parse tree. If trees are tagged and cleaned up (as MSLL does), t can be managed—but it still grows with path count.

10.3. Complexity Summary

Table 9: MSLL vs. LL(1)/LL(K)/ANTLR complexity comparison

Aspect	MSLL	ll(1)/ll(k)	ANTLR (DFA)
Time Complexity	$O(b^d \cdot n)$	$O(n)$	$O(n)avg$
Space Complexity	$O(b^d \cdot s + t)$	$O(s)$	$O(s)avg$
Lookahead	adaptive	fixed (k)	precompiled
Ambiguity Handling	runtime split	rewrite/fail	statically

MSLL trades performance for interactive flexibility. It is not suitable for very large inputs with many conflict points, but is ideal for grammar prototyping, DSL editing, and embedded parsing where grammar is volatile or simple, or performance is secondary(see Table 9).

11. Discussion

The design of MSLL reflects a deliberate trade-off between runtime efficiency and development flexibility. Our evaluation compares MSLL against LL(1) and ANTLR parsers to highlight this trade-off. LL(1) parsers represent a near-optimal baseline for deterministic grammars due to their single-path execution model, while ANTLR's LL($*$) parsers optimize runtime performance through offline DFA construction [1, 3].

MSLL targets a different usage scenario. By executing grammars directly at runtime and resolving ambiguity through controlled stack duplication, MSLL eliminates grammar rewriting and parser regeneration. This design improves transparency and responsiveness during grammar development, at the cost of higher runtime overhead under deep ambiguity. Such trade-offs are consistent with prior observations that generated parsers shift complexity from runtime to development time [6].

MSLL is particularly well suited for environments in which grammars evolve frequently or are authored by end users, such as DSL editors, scripting platforms, and educational tools. In these settings, immediate feedback and debuggability often outweigh raw parsing throughput. Once grammars stabilize, the same `.g4` specifications can be deployed unchanged using ANTLR to achieve production-grade performance.

The approach adopted by MSLL is complementary to incremental parsing systems [19, 20]. While incremental parsers optimize repeated edits to stable grammars, MSLL focuses on rapid experimentation with the grammars themselves. Together, these techniques support different phases of the language development lifecycle.

12. Limitations and Future Work

12.1. Limitations

While MSLL offers a novel and practical approach to runtime parsing for ambiguous grammars, it is not without limitations. These primarily relate to scalability, memory consumption, and suitability for production deployment in performance-critical settings.

- **Stack Explosion Under Deep Ambiguity**: The core mechanism of MSLL—parallel stack duplication for ambiguous productions—can result in an exponential number of active stacks in worst-case scenarios.

As shown in our evaluation (Section VII), deeply nested or recursively ambiguous grammars (e.g., object structures with recursive fields) may trigger the creation of tens of thousands of stacks, leading to increased memory usage and longer parsing times. Although MSLL's pruning and greedy-first optimizations mitigate this to some extent, stack explosion remains a fundamental limitation in high-ambiguity contexts.

- **Lack of Incremental Parsing Support**: MSLL currently performs a full reparse of the input on every invocation. While this is acceptable for small-to-medium inputs in live prototyping environments, it becomes less suitable for real-time feedback in large files or codebases. In contrast, modern parsers like Tree-sitter [19] and Roslyn [20] employ incremental techniques, preserving parse subtrees across edits to reduce recomputation. Integrating incrementalism into MSLL remains an open challenge, particularly in maintaining correctness across multiple active stacks.

- **No Integrated Error Recovery**: When all parsing stacks are pruned due to mismatch or dead ends, MSLL reports a parsing failure without attempting recovery. Unlike robust production parsers that use panic-mode recovery or synchronization tokens, MSLL provides no facilities for generating partial trees or continuing parsing after errors. This limits its usability in editor settings that demand graceful degradation and continuous analysis even with syntax errors.

- **Limited Formal Proof Machinery**: While we provide soundness and completeness sketches for MSLL, a full formalization—including operational semantics, inductive correctness proofs, and ambiguity resolution guarantees—remains to be developed. A mechanized proof (e.g., in Coq or Isabelle) could further strengthen MSLL's theoretical foundation and help identify subtle corner cases in pruning or stack management.

12.2. Future Work

To address these limitations and extend the scope of MSLL, we outline several promising directions:

- **Conflict-Bounded Execution**: By statically estimating a conflict depth bound d, MSLL could retain only the top-k most promising

stacks (based on token match length or grammar heuristics). This would reduce memory usage and prevent pathological explosion.

- **Parallel Stack Evaluation**: Leveraging multi-core systems to evaluate parsing stacks in parallel would significantly reduce wall-clock time, particularly in high-ambiguity workloads. A parallel runtime model with coordinated pruning and shared tree nodes could retain correctness while improving responsiveness.

- **Incremental Stack Persistence**: Future iterations of MSLL could explore reusing partial parse results and tree fragments across invocations, maintaining a cache of successful paths and merging results on small edits.

- **Error Recovery Mechanisms**: Lightweight error recovery techniques (e.g., token deletion, insertion, or rule skipping) can be added to MSLL to enable best-effort parsing in the presence of malformed input, improving its suitability for real-time editing environments.

- **IDE Plugin Integration**: Embedding MSLL into widely used editor frameworks (e.g., VSCode, IntelliJ) as a live parser plugin for DSL development would further validate its utility and uncover real-world usability requirements.

By addressing these areas, MSLL can evolve from a runtime grammar prototyping engine into a versatile parsing framework capable of supporting a broad spectrum of language tools, from exploratory design to educational interpreters and even embedded compilers.

13. Conclusions

This paper presented MSLL, a multi-stack runtime parsing approach that enables interactive handling of ambiguous top-down grammars without DFA construction or parser code generation. Rather than aiming to replace highly optimized production parsers[6] [8], MSLL addresses a different point in the language development lifecycle: grammar design, debugging, and rapid iteration. By executing ANTLR-style grammars directly at runtime and resolving ambiguity via controlled stack duplication, MSLL provides immediate feedback while preserving compatibility with existing LL-based toolchains.

MSLL's primary contribution lies not in outperforming existing parsers in speed, but in offering a more accessible and modifiable parsing workflow. By eliminating the code generation phase, MSLL enables developers to edit .g4 grammar definitions and see parsing results immediately—greatly simplifying the development of custom languages, DSLs, and scripting engines. Its multi-stack architecture provides a clear and intuitive mechanism to explore multiple parsing paths simultaneously, making it well suited for debugging ambiguous or evolving grammars.

While MSLL does incur higher time and space complexity compared to LL(1) or DFA-based parsers [17], this trade-off is acceptable in interactive or embedded environments where responsiveness and flexibility outweigh raw performance. To broaden MSLL's applicability, future work may explore techniques such as bounded stack pruning, multithreaded evaluation, and hybrid integration with static parsers.

In summary, MSLL represents a practical and complementary approach to traditional parser generators. It is best positioned as a prototyping and debugging tool in the early stages of language design or in runtime environments where grammars are frequently modified. For production deployment, grammars refined using MSLL can be migrated to tools like ANTLR to leverage their optimized performance. Together, these tools offer a complete workflow from experimentation to execution, bridging the gap between language prototyping and parser generation.

References

[1] Terence Parr and Kathleen Fisher. LL (*) the foundation of the ANTLR parser generator. *ACM Sigplan Notices*, 46(6):425–436, 2011.

[2] Seppo Sippu and Eljas Soisalon-Soininen. On LL (k) parsing. *Information and Control*, 53(3):141–164, 1982.

[3] Terence Parr, Sam Harwell, and Kathleen Fisher. Adaptive LL (*) parsing: the power of dynamic analysis. *ACM SIGPLAN Notices*, 49(10):579–598, 2014.

[4] A. Gayler Harford, Vincent P. Heurinc, and Michael G. Main. A new parsing method for non-LR (1) grammars. *Software: Practice and Experience*, 22(5):419–437, 1992.

[5] Elizabeth Scott and Adrian Johnstone. GLL parsing. *Electronic Notes in Theoretical Computer Science*, 253(7):177–189, 2010.

[6] William M. Waite and Lynn Robert Carter. The cost of a generated parser. *Software: Practice and Experience*, 15(3):221–237, 1985.

[7] Ladislav Vagner and Bořivoj Melichar. Parallel LL parsing. *Acta informatica*, 44(1):1–21, 2007.

[8] Sam Lasser, Chris Casinghino, Kathleen Fisher, and Cody Roux. A verified LL (1) parser generator. In *10th International Conference on Interactive Theorem Proving (ITP 2019)*, pages 24–1, 2019.

[9] Arnd Rußmann. Dynamic LL () parsing. *Acta Informatica*, 34(4):267–289, 1997.

[10] Stephen A. Blythe, Michael C. James, and Susan H. Rodger. LLparse and LRparse: Visual and interactive tools for parsing. *ACM SIGCSE Bulletin*, 26(1):208–212, 1994.

[11] Norbert Blum. On parsing LL-languages. *Theoretical computer science*, 267(1-2):49–59, 2001.

[12] Gisela Pitsch. LL (k)-PARSING OF COUPLED-CONTEXT-FREE GRAMMARS. *Computational intelligence*, 10(4):563–578, 1994.

[13] Warren X. Li. Building efficient incremental LL parser by augmenting LL table and threading parse tree. In *Proceedings of the 33rd annual ACM Southeast Conference*, pages 269–270, 1995.

[14] Lennart Hjalmarsson, Subal C. Kumbhakar, and Almas Heshmati. DEA, DFA and SFA: a comparison. *Journal of Productivity Analysis*, 7(2):303–327, 1996.

[15] Rajesh Parekh and Vasant Honavar. Learning DFA from simple examples. *Machine Learning*, 44(1):9–35, 2001.

[16] Jens Knoop. From DFA-frameworks to DFA-generators: A unifying multiparadigm approach. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 360–374, 1999.

[17] Terence J. Parr and Russell W. Quong. ANTLR: A predicated-LL (k) parser generator. *Software: Practice and Experience*, 25(7):789–810, 1995.

[18] Bryan Ford. Parsing expression grammars: a recognition-based syntactic foundation. In *Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 111–122, 2004.

[19] Tim A. Wagner and Susan L. Graham. Incremental analysis of real programming languages. *ACM SIGPLAN Notices*, 32(5):31–43, 1997.

[20] John Murray. Introduction to Roslyn: The Compiler as a Service, 2024.

[21] Daan Leijen and Erik Meijer. Parsec: A practical parser library. *Electronic Notes in Theoretical Computer Science*, 41(1):1–20, 2001.